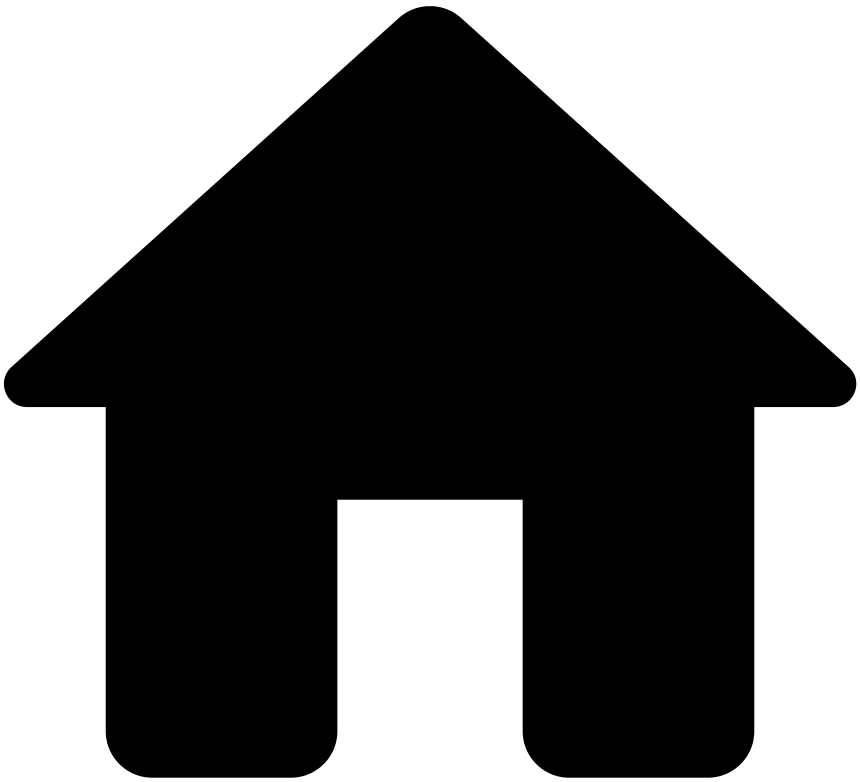
Source: https://cam.feedzai.com/pulse/latest/docs/common/pulse/users-guide/rule-guide/creating-features/using-pql-in-features/pql-in-derived-features



PQL in Derived Features

Was this page helpful?Â 💬

Derived features are new data fields that were calculated based on the original data set.

To create derived features, there are two different cases:

- Computing data based on a single event (not on aggregated data), using a map operator.
- Calculating a value based on an aggregation of events, often combining map and metrics operators.

This topic describes how to use PQL in both cases.

Derived fields without aggregationâ□□

To calculate new values based on input fields, you can use **map operators** in plans. For that, add a map operator to the plan that processes the input data, and add the PQL expression in the *Expression* field.

You can use any of the building blocks of PQL, except for the aggregations.

Exampleâ□□

The following example adds a field called *CountryMatch* that shows if the merchant country is the same as the country where the card was created.

Derived field based on aggregationsâ□□

To aggregate data, use a **metrics operator** in the plan. The UI of the metrics operator allows you to use a shorter list of aggregations: Avg, Min, Max, Count, Count Distinct, Sum and Standard deviation. Other calculations need PQL statements.

PQL allows you to use the following **aggregations** in the *Expression* field of an aggregation when editing a metrics operator:

Aggregation	Description	Usage example
avg(field_name)	Returns the average value of the selected numeric field from the aggregated data set.	`avg(Amount)`
count()	Returns the number of rows in the aggregated data set.	`count()`
max(field_name)	Returns the maximum value of the selected numeric field from the aggregated data set.	`max(Amount)`
maxBy(field_name)	Returns the event containing the maximum value in the given numeric field.	`maxBy(Amount).MerchantId`
min(field_name)	Returns the minimum value of the selected numeric field from the aggregated data set.	`min(Amount)`
minBy(field_name)	Returns the event containing the minimum value in the given numeric field.	`minBy(Amount).MerchantId`
stddev(field_name)	Returns the standard deviation of the selected numeric field's values within the aggregated data set.	`stddev(Amount)`
sum(field_name)	Returns the sum of the selected numeric field values in the aggregated data set.	`sum(Amount)`

Navigate on an aggregated data setâ□□

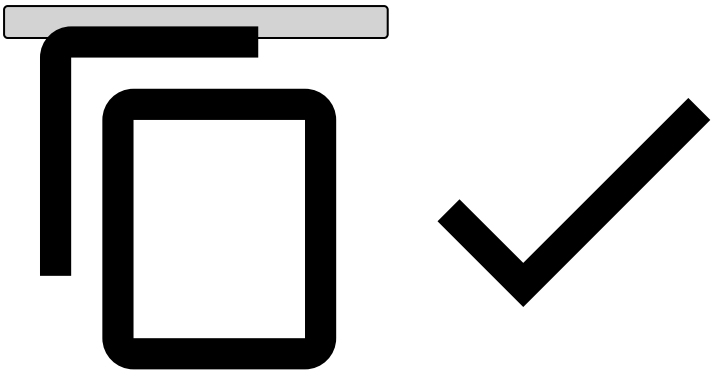
The first(), last(), and prev() methods allow to access the first, last, and previous row of the aggregated data set.

- The current record is the last row of the current dataset if it entered the window, depending on the filters.
- **first()** is the first transaction in the current window.
- **last()** is the last transaction in the current window.
- **prev()** is the transaction before last() that entered the window.

Examplesâ□□

To calculate the geographical distance traveled between two transactions, you can create the following aggregation in a metrics operator:

```
DistanceUtils.haversine(latitude, longitude, prev().latitude, prev().longitude)
```



To calculate the time span between the first and last transaction within a given window per merchant, you can create the following aggregation in a metrics operator:

```
( let first_timestamp = first().Timestamp ?? 0 in
  let last_timestamp = last().Timestamp ?? 0 in
  ( last_timestamp - first_timestamp ) / 60*60*1000.0
)
```